

AI4Math 讨论班

近期动态 · Lean 基本类型与转换

Yiming Fu

目录

- 01 AI4Math 近期动态
- 02 读懂 Lean 代码
- 03 `rfl` 与命题等式
- 04 基本数字类型与转换
- 05 集合的定义
- 06 多项式的定义与计算

联合声明 · 2026-09-11

A Severe Misalignment of AI in Mathematics

The goals of the AI companies and the goals of the mathematical community are severely misaligned.

Tao 参与联署：解题指标与数学理解之间的错位。

AHM · 人类数学协会

The Association for Human Mathematics

工作组：教学、招聘、期刊政策。

成员承诺：不为商业 AI 公司提供服务；不发表 AI 生成的数学文本。

可选 AI-free 承诺：不在数学研究中使用 AI。

MAISI · 数学 AI 安全研究所



Lisa Sakulensky / University of Toronto
Be Giant, 2026-09-10

Canada's star mathematician races to help establish safe superintelligence

An independent non-profit developing mathematical foundations for the safety of powerful AI systems.

科学主任: Jacob Tsimmerman
执行主任: Andrew Critch

读一行定义

```
def twice (n :  $\mathbb{N}$ ) :  $\mathbb{N}$  := n + n
```

```
#check twice      --  $\mathbb{N} \rightarrow \mathbb{N}$ 
```

```
#eval twice 3      -- 6
```

```
#check (3 :  $\mathbb{Q}$ )    -- 3 :  $\mathbb{Q}$ 
```

: 类型 := 定义 #check 看类型 #eval 看值

函数怎么调用

```
def add (a b :  $\mathbb{N}$ ) :  $\mathbb{N}$  := a + b

#check add          --  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 
#check add 3        --  $\mathbb{N} \rightarrow \mathbb{N}$ 
#eval add 3 4       -- 7

#eval (fun n :  $\mathbb{N}$  => n + 1) 4 -- 5
```

`add 3 4` 是两次应用; `fun n => ...` 是匿名函数。

怎样读一个证明

```
#check (2 + 3 = (5 : ℕ)) -- Prop  
theorem two_add_three : (2 : ℕ) + 3 = 5 := rfl  
example (n : ℕ) (h : n = 3) : n = 3 := h  
example (n : ℕ) (h : n = 3) : n = 3 := by  
  exact h
```

$h : n = 3$ 读作我们有 $n = 3$ 的一个证明 h ; $:=$ 后写证明 (值), by 后写 tactic。

显隐式参数：()、{ }

```
def firstExplicit (a : Type) (x _y : a) : a := x
def first {a : Type} (x _y : a) : a := x

#eval firstExplicit ℕ 3 4      -- 3
#eval first (3 : ℕ) 4          -- 3
#eval first (a := ℚ) 3 4       -- 3
#eval @first ℚ 3 4             -- 3
```

$\{a : \text{Type}\}$ 通常可推断; $(a := \mathbb{Q})$ 指定它, @ 将隐式参数显式化。

variable: 公共参数

```
section
variable {α : Type} (x : α)

def keep : α := x
#check keep -- keep {α : Type} (x : α) : α

end

#eval keep (3 : ℕ) -- 3
```

keep 实际仍有参数: $\{\alpha : \text{Type}\} (x : \alpha)$ 。

实例参数： []

```
def addTwo {α : Type} [Add α] (x y : α) : α :=  
  x + y  
  
#eval addTwo (3 : ℕ) 4      -- 7  
#eval addTwo (3 : ℤ) (-4)   -- -1
```

[Add α] 提供加法；Lean 按类型寻找已有实例。

`rfl`: 依定义相等 (`defeq`)

```
example : twice 3 = 6 := rfl
```

```
example (n :  $\mathbb{N}$ ) : twice n = n + n := rfl
```

```
example (n :  $\mathbb{N}$ ) : n + 0 = n := rfl
```

`rfl` 证明的也是 $a = b$; 左右按定义展开、计算后相同。

$0 + n$: 需要一个定理

```
-- example (n :  $\mathbb{N}$ ) :  $0 + n = n$  := rfl
```

```
example (n :  $\mathbb{N}$ ) :  $0 + n = n$  := Nat.zero_add n
```

```
example (n :  $\mathbb{N}$ ) (h :  $n = 3$ ) :  $n + 0 = 3$  := h
```

自然数加法按第二个参数递归; $0 + n$ 不能直接算成 n 。

命题等式：用 h 改写

```
example (n m :  $\mathbb{N}$ ) (h : n = m) : n + 1 = m + 1 := by  
  rw [h]
```

```
-- example (n m :  $\mathbb{N}$ ) (h : n = m) : n = m := rfl
```

$n = m$ 是命题， h 是证明；有 h 不代表 n 、 m 定义相等。

同一个数字，不同类型

```
#check (3 : ℕ)
#check (3 : ℤ)
#check (3 : ℚ)
#check (3 : ℝ)
```

$\mathbb{N}$ 自然数 $\mathbb{Z}$ 整数 $\mathbb{Q}$ 有理数 $\mathbb{R}$ 实数

$\mathbb{N}$: 从 zero 和 succ 开始

```
#print Nat
-- inductive Nat where
--   | zero : Nat
--   | succ : Nat → Nat

example : Nat.succ (Nat.succ Nat.zero) = 2 := rfl
example (n :  $\mathbb{N}$ ) : Nat.succ n = n + 1 := rfl
#eval (0 :  $\mathbb{N}$ ) ^ 0 -- 1
```

`succ n` 表示下一个自然数; `#print` 查看实际定义。

$\mathbb{Z}$: 非负数与负数

```
#print Int
-- ofNat      :  $\mathbb{N} \rightarrow \mathbb{Z}$ 
-- negSucc    :  $\mathbb{N} \rightarrow \mathbb{Z}$ 

#eval Int.ofNat 3    -- 3
#eval Int.negSucc 2  -- -3

example (n :  $\mathbb{N}$ ) : (n :  $\mathbb{Z}$ ) = Int.ofNat n := rfl
```

`ofNat n` 表示 n ; `negSucc n` 表示 $-(n + 1)$ 。

Q: 约分后的分子与分母

```
def q :  $\mathbb{Q}$  := 6 / 8
```

```
#check q.num    --  $\mathbb{Z}$ 
```

```
#check q.den    --  $\mathbb{N}$ 
```

```
#eval q.num     -- 3
```

```
#eval q.den     -- 4
```

```
example : q = 3 / 4 := by norm_num [q]
```

num : $\mathbb{Z}$, den : $\mathbb{N}$; 定义还要求分母非零、分子分母互素。

$\mathbb{R}$: 有理数的完备化

```
example : (1 :  $\mathbb{R}$ ) / 3 + 2 / 3 = 1 := by  
  norm_num
```

```
example (x :  $\mathbb{R}$ ) (hx : x  $\neq$  0) : x / x = 1 := by  
  exact div_self hx
```

```
example : (0 :  $\mathbb{R}$ ) / 0 = 0 := by norm_num
```

Mathlib 用有理数柯西序列的等价类构造 $\mathbb{R}$; 这里只使用它的运算。

$\mathbb{C}$: 实部与虚部

```
def z :  $\mathbb{C}$  := ⟨1, 2⟩

example : z.re = 1 := rfl
example : z.im = 2 := rfl
example : z = 1 + 2 * Complex.I := by
  norm_num [z, Complex.ext_iff]

example (r :  $\mathbb{R}$ ) : (r :  $\mathbb{C}$ ).im = 0 := rfl
```

$\langle 1, 2 \rangle$ 匿名构造子给出实部、虚部, 即 $1 + 2i$; $\mathbb{R} \rightarrow \mathbb{C}$ 的虚部为 0。

已有变量：用 $\uparrow$ 或类型注解转换

```
def toInteger (n :  $\mathbb{N}$ ) :  $\mathbb{Z}$  := n
def toRational (n :  $\mathbb{N}$ ) :  $\mathbb{Q}$  := (n :  $\mathbb{Q}$ )
def toReal (q :  $\mathbb{Q}$ ) :  $\mathbb{R}$  := (q :  $\mathbb{R}$ )

example (n :  $\mathbb{N}$ ) : n = (n :  $\mathbb{Z}$ ) := rfl
```

Lean 也会自动插入已注册的转换实例； $\mathbb{N}$ 与 $\mathbb{Z}$ 仍是不同类型。

先算，还是先转换？

```
#eval ((2 - 3 : ℕ) : ℤ) -- 0
#eval (2 : ℤ) - 3      -- -1

#eval ((7 / 2 : ℕ) : ℚ) -- 3
#eval (7 : ℚ) / 2       -- 7/2

#eval Int.toNat (-3)    -- 0
```

自然数减法、除法会截断

norm_num: 证明具体数值计算

```
example : ((2 - 3 : ℕ) : ℤ) = 0 := by
  norm_num
```

```
example : (2 : ℤ) - 3 = -1 := by
  norm_num
```

```
example : (7 : ℚ) / 2 + 1 / 2 = 4 := by
  norm_num
```

norm_cast: 整理数值转换

```
example (m n : ℕ) :  
  ((m + n : ℕ) : ℤ) = (m : ℤ) + (n : ℤ) := by  
  norm_cast
```

```
example (m n : ℕ) (h : (m : ℤ) = (n : ℤ)) :  
  m + 1 = n + 1 := by  
  norm_cast at h  
  rw [h]
```

可作用于目标，也可用 `norm_cast at h` 处理假设。

exact_mod_cast: 使用转换前后的证明

```
example (m n :  $\mathbb{N}$ ) (h : m  $\leq$  n) :  
  (m :  $\mathbb{R}$ )  $\leq$  (n :  $\mathbb{R}$ ) := by  
  exact_mod_cast h
```

```
example (m n :  $\mathbb{N}$ ) (h : (m :  $\mathbb{Z}$ ) < (n :  $\mathbb{Z}$ )) : m < n := by  
  exact_mod_cast h
```

假设与目标主要差在类型转换时，用 `exact_mod_cast h`。

修正平均值，再验证结果

```
def averageWrong (a b : ℕ) : ℚ := ((a + b) / 2 : ℕ)
def average (a b : ℕ) : ℚ := ((a : ℚ) + (b : ℚ)) / 2

#eval averageWrong 2 3 -- 2
#eval average 2 3      -- 5/2

example : average 2 3 = 5 / 2 := by
  norm_num [average]
```

先转成 $\mathbb{Q}$ 再除；`norm_num [average]` 展开定义并证明结果。

Set α : 用一个命题描述成员

```
example : Set  $\mathbb{N}$  = ( $\mathbb{N} \rightarrow \text{Prop}$ ) := rfl
```

```
def small : Set  $\mathbb{N}$  := {n | n < 5}
```

```
example : small = (fun n :  $\mathbb{N}$  => n < 5) := rfl
```

```
example (n :  $\mathbb{N}$ ) : (n  $\in$  small) = (n < 5) := rfl
```

```
example : 3  $\in$  small := by norm_num [small]
```

Set α 的定义就是 $\alpha \rightarrow \text{Prop}$; $n \in \text{small}$ 就是 $\text{small } n$ 。

全集、空集与列举元素

```
def allNats : Set ℕ := Set.univ
def noNats : Set ℕ := ∅
def chosen : Set ℕ := {1, 2, 3}

example : allNats = (fun _ => True) := rfl
example : noNats = (fun _ => False) := rfl
example (n : ℕ) :
  (n ∈ chosen) ↔ n = 1 ∨ n = 2 ∨ n = 3 := by
  simp [chosen]
```

True 总成立, False 不成立; $\vee$ 表示“或”。

交与并： $\wedge$ 和 $\vee$

```
example (s t : Set ℕ) (n : ℕ) :  
  (n ∈ s ∧ n ∈ t) ↔ (n ∈ s ∧ n ∈ t) := Iff.rfl
```

```
example (s t : Set ℕ) (n : ℕ) :  
  (n ∈ s ∪ t) ↔ (n ∈ s ∨ n ∈ t) := Iff.rfl
```

```
example : 3 ∈ chosen n small := by  
  norm_num [chosen, small]
```

↔ 表示等价， $\wedge$ 表示 “且”； `Iff.rfl` 证明两侧是同一个命题。

$\mathbb{Z}[X]$: 整数系数多项式

```
open Polynomial

def p :  $\mathbb{Z}[X]$  :=  $X^2 + C\ 2 * X + C\ 3$ 

#check p           -- Polynomial  $\mathbb{Z}$ 
#check (X :  $\mathbb{Z}[X]$ ) -- 不定元
#check C (2 :  $\mathbb{Z}$ )  -- 常数多项式

def linear (a b :  $\mathbb{Z}$ ) :  $\mathbb{Z}[X]$  := C a * X + C b
```

多项式由有限个非零系数确定； x 是不定元， $C\ a$ 是常数多项式。

取系数，再代入一个值

```
#check p.coeff 1 -- ℤ
#check p.eval 2 -- ℤ

example : p.coeff 1 = 2 := by simp [p]
example : p.coeff 2 = 1 := by simp [p, coeff_X]
example : p.coeff 5 = 0 := by simp [p, coeff_X]

example : p.eval 2 = 11 := by norm_num [p]
```

`p.coeff k` 取 k 次项系数; `p.eval a` 代入 a ; `simp` 用已有等式化简。

次数与展开

```
example : p.natDegree = 2 := by
  unfold p
  compute_degree!
```

```
example : ((X + 1) ^ 2 : ℤ[X]) = X ^ 2 + 2 * X + 1 := by
  ring
```

```
example (x : ℚ) : (x + 1) ^ 2 = x ^ 2 + 2 * x + 1 := by
  ring
```

`natDegree` 取自然数次数; `ring` 验证环上的恒等式。

map: 把整数系数换成有理数系数

```
def pQ :  $\mathbb{Q}[X]$  := p.map (algebraMap  $\mathbb{Z}$   $\mathbb{Q}$ )  
  
example : pQ =  $X^2 + C\ 2 * X + C\ 3$  := by  
  simp [pQ, p, C_ofNat]  
  
example : pQ.eval (1 / 2) = 17 / 4 := by  
  norm_num [pQ, p]
```

map 逐个转换系数; algebraMap $\mathbb{Z}$ $\mathbb{Q}$ 给出整数到有理数的映射。